Корзина ещё не резервирует товар

Выбор покупателя и наличие на складе могут измениться независимо.

01
Аня выбрала
Размер 101 × 2

→

02
Прошло время
Другой покупатель купил пары

→

03
Аня оформляет
Остаток проверяем снова

cart_items.quantity
Сколько пар хочет купить человек.

product_sizes.stock
Сколько пар доступно прямо сейчас.

↳ Добавление в корзину не обещает наличие. Окончательное решение принимается при оформлении.

Сервер собирает данные для покупки

Цена и владелец приходят из доверенных серверных источников.

Что нужно	Откуда берём
Пользователь	Проверенный токен → users
Выбранные размеры и количество	Его строки cart_items
Текущий остаток	product_sizes.stock
Цена одной пары	products.price

Тело запроса · опасная подмена — Учебный пример

```
{"user_id": 8, "total": "1.00"}
# Эти значения не назначают
# владельца и сумму заказа
```

Операция оформления

Берём текущего пользователя.
Сумму считаем сами.

↳ Клиент просит оформить свою корзину. Сервер определяет владельца, состав и итог.

Перед записью проверяем условия

Оформление допустимо только для целой корректной корзины.

1. Пользователь установлен	Работаем только с его корзиной
2. Корзина не пустая	Есть хотя бы одна позиция
3. Количество допустимо	Для каждой строки quantity > 0
4. Размер существует	Ссылка ведёт к нужному товару
5. Остатка хватает	Для каждой позиции stock ⩾ quantity

↳ Если не проходит хотя бы одна позиция, весь заказ отклоняется без частичного списания.

09 Оформление заказа

Сумму считаем по каждой позиции

Количество умножаем на цену одной пары, затем складываем результаты.

Размер ID	Цена из products	Количество	Стоимость позиции
101	7 990.00	2	15 980.00
201	5 990.00	1	5 990.00

15 980.00 + 5 990.00 **21 970.00 ₽**

`total = Σ(price × quantity)` · точные Decimal / numeric

↳ Все цены — из базы. В этом примере нет доставки, скидок и налогов.

09 Оформление заказа

Заказ запоминает цену покупки

История не пересчитывается при изменении каталога.

Во время покупки

product 10 → **7 990 ₽**
Сохраняем в price_at_purchase.

Позже в каталоге

products.price → **8 990 ₽**

Сохранённая позиция заказа — Учебный пример

```
product_size_id: 101
quantity: 2
price_at_purchase: 7990.00
```

История покупки

Стоимость позиции: **15 980 ₽**
Итог заказа: **21 970 ₽**

↳ price_at_purchase — снимок цены одной пары. total фиксирует согласованный итог заказа.

09 Оформление заказа

Транзакция объединяет изменения

Заказ, его состав, остатки и корзина должны измениться согласованно.

ОДНА ТРАНЗАКЦИЯ

`orders`	`order_items`	`product_sizes`	`cart_items`
Создать заказ	Записать состав и цены	Списать пары	Очистить свою корзину

COMMIT

Все изменения зафиксированы.

ROLLBACK

Изменения этой операции отменены.

↳ Не допускаем «заказ есть, а склад не списан» или «корзина пуста, но заказ не создан».

Оформление по шагам

Текущий пользователь уже установлен. Одна операция владеет транзакцией.

Псевдокод · последовательность — Учебный пример

```
начать транзакцию
заблокировать владельца корзины
прочитать его корзину
заблокировать размеры; прочитать цены
проверить все позиции; посчитать итог
создать заказ и позиции
списать остатки; очистить корзину
зафиксировать; при ошибке откатить
```

Граница одна

Внутренние функции не делают собственный commit.

После ожидания читаем актуальное

Проверяем остаток уже после получения блокировки.

↳ Сначала согласованное чтение и проверки, затем записи. Успех возвращаем после commit.

09 Оформление заказа

После commit виден целый заказ

Сравним состояние до и после оформления корзины Ани.

Что проверяем	До	После commit
Новый заказ	Нет	№ 502 · created
Новые позиции	0	2 · цены 7990.00 и 5990.00
Остатки 101 / 201	3 / 5	1 / 4
Корзина Ани	2 позиции	Пустая
Итог нового заказа	—	21970.00

✓ Проверено в PostgreSQL. Корзина Бориса и прежний заказ 501 сохранены.

↳ Проверяем не только результат функции: читаем заказ, состав, остатки и корзину из базы.

09 Оформление заказа

Ошибка посередине: полный откат

Сбой специально вызван после уже выполненных изменений.

Место ошибки	Новых заказов	Корзина Ани
После INSERT orders	0	2 строки
После списания первой позиции	0	2 строки
После DELETE корзины	0	2 строки

Во всех трёх опытах остатки восстановлены: 101 → 3, 201 → 5.

↳ ROLLBACK отменяет изменения текущей операции даже после INSERT, UPDATE и DELETE.

09 Оформление заказа

Двое увидели последнюю пару

Две отдельные транзакции читают stock=1 до чужого commit.

Шаг	Аня · соединение A	Борис · соединение B
1	Читает stock = 1	Читает stock = 1
2	В памяти: 1 − 1 = 0	В памяти: 1 − 1 = 0
3	Пишет stock = 0, создаёт заказ	Ждёт UPDATE, затем тоже пишет 0
4	COMMIT · покупка прошла	COMMIT · покупка тоже прошла

Была 1 пара → продали 2 В базе stock = 0. Даже CHECK(stock ⩾ 0) не заметил проблему.

↳ Каждая транзакция может быть атомарной и всё равно принимать решение по устаревшему чтению.

Блокируем строку и проверяем снова

Один покупатель завершает работу с остатком, второй ждёт.

Шаг	Аня · первая получила строку	Борис · ждёт ту же строку
1	SELECT … FOR UPDATE → stock=1	SELECT … FOR UPDATE → ожидание
2	Проверяет 1 ⩾ 1; списывает до 0	Ещё не принимает решение о покупке
3	COMMIT · блокировка снята	Получает актуальный stock=0
4	Один новый заказ сохранён	Отказ: остатка не хватает

Блокирующее чтение внутри транзакции — Учебный пример

```
SELECT id, stock FROM product_sizes
WHERE id = 101 FOR UPDATE;
```

↳ В опыте READ COMMITTED: второй покупатель после ожидания видит 0 и не создаёт заказ.

Проверяем границы защиты

Последняя пара — только один из обязательных сценариев.

Сценарий	Проверенный результат
Пустая / не хватает второй позиции	Отказ, изменения не сохраняются
Два оформления одного пользователя	Один заказ; второе видит пустую корзину
Изменение корзины ждёт оформление	Новая позиция остаётся после commit

> Потерянный ответ после commit — отдельный случай: повтор запроса требует политики идемпотентности.

↳ Все операции одной корзины должны соблюдать общий протокол. Блокировки склада для этого недостаточно.

Практика: собираем магазин по модулям

После каждого модуля должен появиться результат, который можно проверить.

M0
Окружение
/docs и запрос к БД

M1
Схема и данные
Миграции + seed

M2
Каталог
Чтение, фильтры, CRUD

M3
Вход и права
JWT + защита изменений

M4
Корзина
Данные текущего пользователя

M5
Заказы
Сумма, остатки, транзакция

M6
Завершение
Проверки + README + сдача

↳ Движемся M0 → M6. Один пройденный модуль становится опорой для следующего.

Один модуль делим на малые задачи

Пример M0: четыре шага, у каждого свой проверяемый выход.

1. Python и venv — Файлы зависимостей

Нужный интерпретатор; пакеты установлены

2. PostgreSQL в Compose — `compose.yaml, .env.example`

Контейнер готов; SQL выполняется

3. Каркас FastAPI — `app/main.py,` настройки

Открывается /docs

4. Подключение и /health — `database.py,` обработчик

200 при доступной БД; 503 при сбое

↳ Следующую задачу начинаем после проверки текущей. Размер шага — один понятный результат.

Промпт описывает задачу и проверку

Помощнику нужны конкретные условия текущего проекта.

Цель

Какое поведение должно появиться

Контекст

Что уже есть и какие файлы приложены

Ограничения

Стек, права, границы изменения

Результат

Какие файлы и diff ожидаем

Приёмка

Команды, ответы и состояние данных

Объяснение

Почему решение работает

↳ «Добавь настоящий /health и проверь отказ БД» задаёт более проверяемую цель, чем «сделай всё хорошо».

Готовый промпт для одного шага

Задача M0: /health подтверждает настоящее подключение к базе.

Короткий вариант · полный текст в HTML — Учебный пример

```
Добавь GET /health в текущий FastAPI.
Используй существующий Engine.
Выполни SELECT 1 с таймаутами.
БД доступна → 200, status: ok.
Ошибка доступности → 503.
Покажи diff и команды проверки.
```

Что приложить

main.py · database.py · настройки
requirements.txt · compose.yaml

Что принять

Рабочая БД → успех.
Учебная БД недоступна → отказ.
После восстановления → успех.

↳ Это условия будущей реализации M0. Возврат постоянного {status: ok} их не выполняет.

Передаём нужный контекст

Помощник должен видеть текущие файлы и воспроизводимый сценарий.

Код и договорённости

Дерево проекта · относящиеся файлы
Версии · контракт API · ограничения

Как воспроизвести

Рабочая папка · команда запуска
Запрос · ответ · состояние БД

Пример обезличенного контекста — Учебный пример

```
API: Python на компьютере
БД: сервис db в Docker
Хост: 127.0.0.1:55432
DATABASE_URL: <REDACTED>
JWT_SECRET: <REDACTED>
Токен: <REDACTED>
```

↳ Показываем структуру и существенные настройки. Реальные пароли, ключи и токены заменяем.

10 Практика с нейросетью

Читаем diff до запуска

Проверяем, что новая строка выполняет нужное действие.

Упрощённый фрагмент diff · идея — Учебный пример

```
 def health():
+    with engine.connect() as conn:
+        conn.execute(text("SELECT 1"))
     return {"status": "ok"}
```

Что изменилось

Появился настоящий SQL-запрос.
Соединение закрывается через with.

Чего фрагмент ещё не доказывает

Откуда engine и text?
Где таймауты и ответ 503?

↳ Читаем не только добавленные строки: проверяем импорты, вызовы, удалённое поведение и ошибки.

Проверка заканчивается наблюдением

Команда запуска, ответ и данные должны подтверждать одно поведение.

01
Папка
Где выполняю команду

→

02
Запуск
Что именно запускаю

→

03
Ответ
Статус + тело

→

04
Данные
Что изменилось в БД

Воспроизводимый опыт из этой главы — Учебный пример

```
cd presentation/sneaker-checkout-examples
python verify.py
```

Фактический результат опыта

23 проверки пройдены.
Откаты, гонки, корзина и остатки проверены в PostgreSQL.

↳ Условия установки и запуска — в инструкции примеров. Один код ответа не доказывает все изменения.

10 Практика с нейросетью

Сообщаем об ошибке воспроизводимо

Нужно объяснить разницу между ожиданием и фактом.

Карточка проблемы · пример — Учебный пример

```
Шаг: M0, проверка /health
API на компьютере; БД в Docker
Ожидал: 200, {status: ok}
Получил: 503, connection refused
Адрес: 127.0.0.1:5432
Публикация БД: 55432 → 5432
```

Проверяем гипотезу

Порт в адресе API отличается от опубликованного порта компьютера.

После исправления

Повторить исходный запрос.
Проверить отказ и восстановление БД.

↳ Описываем сценарий, ошибку и относящийся код. Исправляем причину, затем повторяем проверку.

10 Практика с нейросетью

Отдельное ревью ищет пропущенное

Успешный запрос Ани ещё не доказывает защиту её заказа.

Условие выборки после исправления — Учебный пример

```
select(Order).where(
    Order.id == order_id,
    Order.user_id == user.id,
)
```

Повторная проверка

Аня → заказ 501: **200**
Борис → заказ 501: **404**
Без действительного токена: **401**

↳ Ревью сверяет код с требованиями и контрпримерами. После исправления повторяем проверку двумя аккаунтами.

Переходим дальше после приёмки

Нужно уметь запустить результат, проверить его и объяснить.

Работает

Положительный сценарий воспроизведён

Отказывает правильно

Граничный случай не ломает данные и права

Понятно

Могу объяснить путь запроса и новую часть кода

Сохранено

Diff просмотрен; код и результаты зафиксированы

Короткая запись о шаге	Что сохранить
Цель и файлы	Что изменили и зачем
Команды и факт	Что запустили; что получили
Осталось	Непроверенное и следующий маленький шаг

↳ Следующая глава — практика M0: начинаем с окружения и доводим подключение к БД до проверяемого результата.